Volume 34

Sentinel Object Kernel (SOK)

Status: Core Architecture

Priority: Critical

Vision

Project Sentinel is **not** built around databases.

It is **not** built around scans.

It is **not** built around reports.

It is built around **Objects**.

Everything is an Object.

Everything.

Universal Object Model

Every entity inside Project Sentinel inherits from the same base model.

```
Sentinel Object
```

This means there are **no special cases**.

A laptop behaves like a website.

A website behaves like an employee.

An employee behaves like a cloud account.

Only their properties differ.

Object Philosophy

Every object answers exactly the same questions.

Who am I?

What am I?

Who owns me?

What am I connected to?

What happened to me?

What evidence exists?

What missions involve me?

Which policies affect me?

How trusted am I?

When was I last verified?

Universal Object ID

Every object receives a permanent ID.

Example

```
PS-OBJ-000000000000248391
```

This ID never changes.

Even if:

- hostname changes
- employee changes
- IP changes
- department changes
- cloud provider changes

The object remains the same.

Object Identity Layer

Every object contains

Internal ID

Global UUID

Object Type

Object Version

Creation Time

Current State

Owner

Tenant

Classification

Verification Status

Object Types

Examples

Website

API

Endpoint

Server

Cloud Instance

Container

Kubernetes Cluster

Mobile Device

Employee

Department

Identity

Certificate

DNS Zone

Email Domain

Repository

Source Code

AI Model

Mission

Workflow

Policy

Evidence

Rule

Finding

Case

Incident

Report

Everything uses one engine.

Object Lifecycle

```
Created
```

Every lifecycle transition generates an event.

Object Relationships

Instead of foreign keys.

We build relationships.

Example

```
Employee
```

The Knowledge Engine reasons over these relationships.

Object Snapshots

Every object periodically creates immutable snapshots.

Example

```
Laptop
```

This allows comparison across time without overwriting history.

Object State Engine

The kernel maintains the current state derived from:

- latest evidence
- validated events
- active policies
- verified relationships

Historical information remains preserved separately.

Object Versioning

Objects evolve.

```
Version 1
```

Versions support auditability and rollback of metadata where appropriate.

Object Labels

Objects can have labels.

Examples

Finance

Production

Critical

PCI

ISO27001

GDPR

External

Internal

Cloud

Remote Worker

Labels simplify filtering and automation.

Object Tags

Unlike labels,

Tags are user-defined.

Examples

Migration2028

Acquisition

RedTeam

VIP

Temporary

Tags support organization-specific workflows.

Object Classification

Every object has a classification.

Examples

Public

Internal

Confidential

Restricted

Highly Restricted

Policies can use these classifications.

Object Ownership

Every object always has an owner.

Could be

Person

Department

System

Automation

Unknown

Ownership supports accountability.

Object Trust

Trust belongs to the object.

Not to reports.

Not to scans.

The object maintains its own trust history.

Object Evidence

Evidence never belongs to a report.

Evidence belongs to the object.

Reports merely reference evidence.

This avoids duplication.

Object Timeline

Everything appears on the timeline.

Example

`Created`

Object Memory

One of the platform's most powerful ideas.

Every object remembers.

Not just:

Current State

But

Everything that ever happened.

This creates organizational memory.

NEW CORE ENGINE

Object Registry

Think of this as the "directory" of the platform.

Responsibilities:

- Register new objects.
- Prevent duplicates.
- Resolve identities.
- Track lifecycle.
- Assign IDs.
- Maintain references.

Nothing exists outside the registry.

NEW CORE ENGINE

Object Resolver

Sometimes different data sources describe the same asset.

Example:

```
Laptop-123
```

The Resolver correlates these into one canonical object.

NEW CORE ENGINE

Object Relationship Engine

Responsible for maintaining relationship integrity.

Questions it answers:

- Which assets depend on this object?
- Which identities can access it?
- Which missions include it?
- Which compliance controls reference it?
- Which evidence supports it?

NEW CORE ENGINE

Object Cache

Frequently accessed objects are cached in memory.

Benefits:

- Faster dashboards.
- Faster searches.
- Lower resource usage.
- Better scalability.

The cache never becomes the source of truth.

NEW CORE ENGINE

Object Integrity Engine

Every object receives:

- Integrity Hash.
- Version Signature.
- Verification Status.
- Change Chain.

This supports auditing and tamper detection.

NEW IDEA

Object Explorer

Instead of browsing folders, users explore relationships.

Example:

```
Organization
```

Users navigate the environment naturally.

NEW IDEA

Object Time Travel

Select any date.

The kernel reconstructs the object's known state using recorded events and evidence.

This is invaluable for:

- incident investigations,
- compliance audits,
- forensic reviews,
- change analysis.

NEW IDEA

Object Health

Every object exposes health indicators.

Example:

```
Identity
```

Different object types expose different health dimensions while sharing the same overall model.

PS-ADR-0029

Object-Centric Architecture

Status: Accepted

Decision

Project Sentinel shall model every managed entity as a Sentinel Object with a consistent identity, lifecycle, relationship model, evidence chain, and event history.

Rationale

A universal object model reduces complexity, simplifies extensibility, and allows all platform capabilities to operate on a common foundation.

New Long-Term Vision

Sentinel Cyber Operating Kernel (SCOK)

I think we can now see the architecture as a layered kernel:

```
Object Kernel
```

Everything above the kernel is replaceable.

The kernel remains stable.

One architectural idea that could become one of Project Sentinel's signatures

I'd like to introduce the concept of **"Living Objects."**

A Living Object is not just a record.

It continuously evolves as new evidence arrives:

- It gains new relationships.
- It updates its trust.
- It accumulates history.
- It participates in missions.
- It contributes to organizational knowledge.

In other words, the platform does not merely store information—it maintains a continuously evolving understanding of every authorized asset in the environment.

Looking Ahead

After the Object Kernel, I believe the next foundational volume should be **Volume 35 – Sentinel Query & Discovery Language (SQDL).**

Rather than relying on traditional SQL-style thinking, we can design a domain-specific language tailored to cybersecurity. It would allow users to ask questions like:

- "Show all Finance laptops with declining trust."
- "List externally exposed APIs without recent verification."
- "Find all assets related to this incident."
- "Show every object changed during the last 24 hours."

Because SQDL would operate directly on our Object Kernel and Knowledge Graph, it could become one of the defining capabilities of Project Sentinel—making complex cyber environments searchable in a way that is intuitive, powerful, and consistent with everything we have designed so far.

=====[USER]=====

Good and continue

=====[ASSISTANT]=====

Excellent. We have now reached what I believe is one of the biggest differentiators that can make **Project Sentinel** unique.

Almost every security platform today expects users to **learn the product**.

I think we should reverse that.

The product should learn how the user thinks.

This is completely achievable and doesn't require "magic AI." It requires good software architecture.

PROJECT SENTINEL BLUEPRINT

Volume 35